An Agile and Accurate Energy Estimation Methodology for CGRA-Mapped Algorithms: The *Synchoros* Approach

Ritika Ratnu, Dimitrios Stathis and Ahmed Hemani
KTH Royal Institute of Technology
Stockholm, Sweden
{ratnu,stathis,hemani}@kth.se

Abstract

Coarse-Grained Reconfigurable Arrays (CGRAs) are promising as accelerators for energy and performance benefits. Agile and accurate energy estimates are essential to explore the CGRA design space and guide algorithm mapping decisions. Existing estimations are agile but inaccurate since they do not factor in the impact of wires in the design. This paper presents *synchoros* energy estimation methodology for post-route accurate estimations. The methodology leverages the properties of synchoros VLSI design style to incorporate the impact of interconnects in a CGRA design. Overall, our methodology achieves 98% estimation accuracy relative to post-route results for algorithms irrespective of variations in functionality, dimensions, and parallelism.

1 Introduction

Coarse-Grained Reconfigurable Arrays (CGRAs) are promising accelerators that balance flexibility and energy efficiency for accelerating compute-intensive algorithms. CGRAs feature an array of Processing Engines (PEs), each with compute and data memory elements connected through a configurable interconnect network. Design Space Exploration (DSE) for CGRAs consists of evaluating multiple micro-architectural options and tiling strategies to support algorithms that vary in: 1) functionality, or the types of algorithms, (2) data dimensions, and (3) parallelism— the number of PEs and ports for memory access.

DSE requires agile and accurate energy estimates: Energy is a key criterion for DSE. Energy estimations at DSE must be agile to evaluate desired design space points in in a reasonable time and accurate to avoid misleading results. However, accurate energy consumption of the design is available only after it is physically synthesized, i.e., when the design logic is implemented with standard cells and placed and routed (PnR) with wires (see Figure 1). The energy resulting from routing of wires is significant—up to half of the total dynamic power, and becoming increasingly challenging to estimate as technology scales down [4, 9, 18, 24].

State-of-the-art estimators are agile but inaccurate: As physical synthesis is time-consuming, it is not scalable during DSE. Existing estimators favour agile methods that overlook the impact of layout and wires in the design, compromising accuracy. Accelergy [23] is a popular estimation methodology for domain-specific accelerators (DSAs) that relies on fine-grained component characterization. Fine-grained components are the microarchitectural units at the finest granularity, e.g., an adder, that are common across most designs. This makes the methodology broadly applicable, but its accuracy is limited as it does not account for the dominant energy of inter-component and inter-PE wires, which is only known after completing physical design. Examples of fine-grained characterization also include Aladdin [17], DSAGEN [20], and X-CGRA [1].

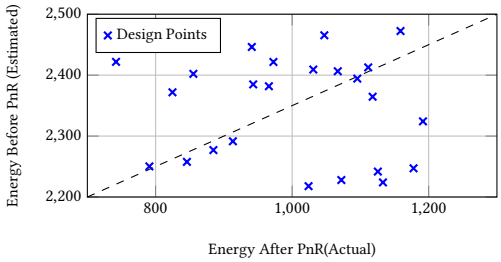

Figure 1: Pre-PnR and post-PnR energy consumption comparison for a 4x4 DRRA [8, 15] CGRA.

Other estimators characterize specific instances from the design space after synthesis [3, 10, 22]. Shi et al. [18] propose a meet-in-the-middle approach that characterizes place-and-routed PEs and uses a neural network model to estimate the impact of inter-PE wires. While these approaches can factor wires into characterization, *they are limited to specific design instances*. Estimates are inconsistent for variations in functionality, dimensions, and parallelism.

Preserving physical information at DSE is the answer: We note that the ad-hoc placement and routing between standard cells during physical synthesis imparts uncertainty in modelling energy consumption. *Synchoros* (from Greek word "choros" for space) VLSI design style [8] addresses this fundamental limitation of standard cell–based design flows: true implementation cost is known only after physical design. The broader objective of synchoricity has many sub-problems, and in this paper, we focus on one such sub-problem: predicting the energy cost of CGRA designs with agility and post-route accuracy.

Contributions. In this paper, we present a synchoros energy characterization and estimation methodology [1] that (1) enables a scalable one-time characterization effort that naturally factors in the impact of interconnects in a customizable, heterogeneous CGRA fabrics, and (2) provides agile and accurate estimations for diverse algorithms irrespective of variations in functionality, dimensions, and parallelism.

DAC '26, Long Beach, CA, USA

ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804396

[1] https://github.com/ritikaratnu111/synchoros-energy-estimation.git

2 Background and Motivation

In this section, we present a background of synchoros VLSI Design style and the motivation for leveraging synchoricity in estimation.

2.1 Synchoros VLSI design style

Synchoros VLSI design style aims at preserving the physical design details at all levels of abstraction. *Synchoricity* discretises space with a virtual grid and uses SiLago (Silicon Lego) blocks as coarse-grained building units, like Lego bricks. The blocks absorb all the metal layers' details so that a CGRA can be composed by abutment of SiLago blocks without having to do logic or physical synthesis. The core idea is to ensure that all valid neighbours bring their wires to the right place on the periphery and the right metal layer and incorporate ancillary structures and circuits to ensure design rules are not violated. This enables composing arbitrary synchoros designs in terms of SiLago blocks to create timing and DRC clean GDSII [7, 19]. Wires are not synthesised de novo, as in conventional standard-cell–based flows. Instead, wires emerge through the abutment of pre-existing wire fragments in SiLago blocks.

SiLago blocks, like Lego bricks, come in different types to cater to different requirements and fall under two main categories, infrastructural and functional. Infrastructural blocks are used to glue together functional regions. Functional regions are aggregates of functional blocks for acceleration, such as CGRAs (Figure 2).

The concept of chiplets [12] has similarities; however, the composition of chiplets is in terms of manufactured chips, whereas in the case of SiLago blocks, it is the GDSII-level designs that abut to create a design. Similar principles for modular design have been discussed by Dally et al. [5]. The key difference is that Dally et. al are attempting to streamline the backend VLSI design flow for efficiency. The objectives of synchoricity aim at automating design from higher abstractions by making the impact of wires predictable.

2.2 Motivation for synchoros energy estimation

Let O denote the set of all operations supported by a CGRA. For an algorithm mapping m, the CGRA configware specifies a subset,

$$O_m \subseteq O$$

consisting of operations that are actually invoked. For each operation $o_i \in O_m$, the configware also specifies: • its spatial context, and • its activation count n_i.

The total energy E_m of the mapped instance is:

$$E_m = \sum_{o_i \epsilon O_m} n_i \cdot e_i$$

where e_i is the energy consumed per activation of o_i. The value e_i is an intrinsic property of the operation but also depends on its *spatial context*, i.e., the set of neighbouring PEs. Different neighbours imply variations in wire length, drive strength, and load, all of which affect energy consumption.

In synchoros design, the regularity of the grid ensures space invariance of the spatial context. When two valid neighbours abut at any place in the fabric, the electrical and technological implications of this abutment are identical. Consequently,

- if operations o_i and o_j are mapped to different PEs, then $e_i = e_j$

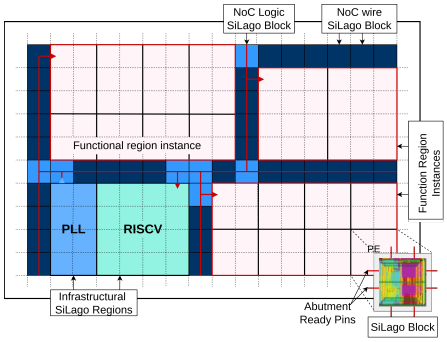

Figure 2: Example floorplan of a synchoros design. Three CGRA functional regions are composed at the GDSII level by abutment of SiLago blocks on a virtual grid.

- for any modifications in algorithm functionality, dimensions, and parallelism, only the set O_m and n_i change; the characterized values e_i remain valid.

These properties enable not only agile, post-route-accurate estimation but also a scalable one-time characterization effort. Spatial invariance does not hold in asynchoros standard cell-based design flows, where physical design is done de novo. Consequently, identical operations at different coordinates may yield different energy values. This could imply two cases. One, storing in an unscalably large database with energy values for all possible variations in functionality, dimensions, and parallelism. Two, profiling only a subset of values at the cost of accuracy.

In contrast, a one-time characterization of a SiLago block in the context of its valid neighbors is sufficient for post-route accurate estimations for any algorithm mapping. The one-time process is hidden from the end user, who only uses the database of characterised components. This is similar to a one-time characterization of standard cells; however, standard-cell characterization does not include characterization of the wires connecting them, whereas our methodology inherently accounts for the impact of wires.

3 Synchoros Characterization Methodology

In this section, we describe the proposed characterization methodology that leverages the synchoricity and its spatial invariance to achieve a scalable, one-time engineering effort to factor the impact of all wires in the design.

Mapping and Configware Assumptions. We assume that when an algorithm is mapped onto a customizable, heterogeneous CGRA fabric, the mapper (also referred to as a compiler/synthesis tool) produces the resulting configware. This configware fully specifies the spatio-temporal distribution of operations executed by CGRA cells. In practice, CGRA mappers commonly produce configware as a critical output for functional verification [6, 16, 21, 25].

3.1 Characterization Building Blocks

The starting point for characterization is a set of abutment-ready SiLago blocks. The blocks correspond to the CGRA PEs in a heterogeneous, customizable CGRA framework. Some CGRA frameworks, however, could be homogeneous and would constitute a subset of the assumed framework. The blocks are synthesized as Design Rule Check (DRC) and timing-clean GDSII macros. The mapper

determines the number and types of blocks required to implement a given mapping and also enumerates the operations for each block.

Each operation is bound to specific *functional resources* within the corresponding SiLago block—for example, ALUs to perform basic arithmetic and logic operations, or a register file (RF) for temporary data storage. These resources get explicitly identified by the operations specified by the mapper. Based on the CGRA design goal, the block may include more sophisticated functional resources, such as address generators (AGs), configured to access external memory in specific patterns. Overall, the number of functional resources is usually limited and can be profiled for characterization.

In a CGRA, Network-on-Chip (NoC) serves as the primary functional interconnect for data movement, while the clock tree (CT) serves as the infrastructural interconnect for clock distribution. In synchoros CGRAs, each SiLago block includes a local NoC or *LNoC interconnect resource* and a local CT or *LCT interconnect resource*. The final CGRA interconnect is composed by the abutment of local interconnect segments. LNoC consists of switchboxes, buffers, and registers for pipelining. While LNoC is not explicitly identified by the configware, it is implied by the data-movement schedule dictated by the operations. LCT is not identified or implied in operations but contributes to energy consumption and, hence, is also characterized.

3.2 Spatial-Context Aware Characterization

A SiLago block is characterized in the context of its valid neighbours. Valid neighbours include the span of PEs that are directly connected to each other. This enables to account for accurate drive strength and capacitive loads. For this purpose, a minimal SiLago design is generated by abutment. Consequently, the impact of all intra- and inter-block wires is inherently factored into the characterization. The set of valid neighbours is often much smaller than the total number of PEs in the design (see Figure 3).

The energy cost of intra-component and inter-component wires is known because a SiLago block is characterized after the synthesis of the minimal design, after the locations of all transistors and wire segments within the block are fixed. As a result, the electrical impact of internal wires—both inside and between functional components—is included in the characterization of each resource.

The CGRA NoC and CT are composed by the abutment of pre-existing LNoC and LCT segments. When two neighboring blocks abut, the corresponding wire segments align to create a larger, valid design without requiring re-synthesis or parasitics extraction. Because each block is characterized in the context of its valid neighbors, inter-block electrical effects—such as variations in drive strength or capacitive loading—are inherently captured.

3.3 Characterization Algorithm

Within the minimal design, the individual block serves as the DUT(Design-Under-Test) for characterization. The block is simulated with the possible operations it supports. Configware information is used to profile the raw toggling activity of its resources during these operations. By analyzing the toggling patterns from simulations, we can determine power consumption for each operation and its associated hardware. The entire process results in a database of average power consumption for each block across

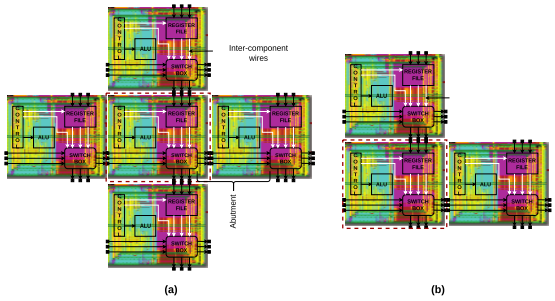

Figure 3: Example of two valid-neighbor configurations for synchoros characterization.

all possible operating modes. Importantly, these power values account for all wires from the fabric level down to the inter-standard cell. The characterization algorithm for preparing the database of characterized operations is presented in Algorithm 1.

To obtain accurate power values for every operation supported by the resources in a SiLago block, we employ an iterative characterization flow with a convergence criteria across all contributing functional and interconnect components. For each block type $s \in S$, the algorithm enumerates all legal neighborhood configurations $N \subseteq \text{ValidNeighbors}(s)$ and generates the corresponding SiLago design $D = \text{Abutment}(s, N)$. For each operation $op \in \text{Ops}(s)$ and each internal resource $r \in \text{Resources}(op)$, the instantaneous power P_{inst} is repeatedly measured through post-layout parasitics back-annotated gate-level power simulation of D. The running mean $P_{r,op}$ is updated as the online mean of all collected samples:

$$P_{r,op}^{(k+1)} = \frac{k\, P_{r,op}^{(k)} + P_{\text{inst}}^{(k+1)}}{k+1}. \tag{1}$$

As gate-level simulations are time-consuming, we separate the characterization of static power consumption, which requires a single simulation iteration, from the dynamic power simulation, which requires multiple iterations. To minimize the number of iterations required to characterize dynamic power consumption, we have adopted a convergence-based strategy, as shown in Figure 4. To ensure that the characterization does not introduce bias, random stimuli are applied, and a running average is maintained. After a minimum number of simulations is performed, the error between consecutive running averages of power is tracked over a window of simulations. If the errors in all of the simulations within the window are below the convergence threshold, the simulations are terminated. The threshold and the simulation window are user-defined.

To ensure stability across varying inputs, convergence is not declared immediately. Instead, after an initial settling period of n iterations, the algorithm tracks the absolute deviation between the latest sample and the running mean,

$$e^{(k)} = \left| P_{\text{inst}}^{(k)} - P_{r,op}^{(k)} \right|, \tag{2}$$

and stores it in a sliding window of size w. Convergence is reached only when all errors in the window satisfy

$$e^{(j)} < \epsilon \qquad \forall j \in \{k - w + 1, \ldots, k\}. \tag{3}$$

Algorithm 1: Power extraction with convergence criteria for all components

Data: SiLago Blocks S, set $N \subseteq$ ValidNeighbors(s), convergence threshold ϵ, window size w, minimum iterations n
Result: Characterized power $P_{r,op}$ for each operation $op \in Op(S)$
foreach *SiLago Block Type* $s \in S$ **do**
  foreach *set* $N \subseteq$ *ValidNeighbors*(s) **do**
    Generate SiLago design $D \leftarrow$ Abutment(s, N);
    foreach *operation* $op \in Ops(s))$ **do**
      foreach *resource* $r \in \mathcal{R}(op)$ **do**
        Initialize $P_{r,op} \leftarrow 0$, $k \leftarrow 0$;
        Initialize error array $error[w] \leftarrow$ empty;
        $converged \leftarrow$ false;
        while *not converged* **do**
          $P_{inst} \leftarrow getPower(D, r, op)$;
          $P_{r,op} \leftarrow \frac{k \cdot P_{r,op} + P_{inst}}{k+1}$;
          $k \leftarrow k + 1$;
          if $k > n$ **then**
            $e \leftarrow |P_{inst} - P_{r,op}|$;
            Append e to $error$;
            if *len(error)* $> w$ **then**
              Remove oldest entry from $error$;
            end
            if *len(error)* $= w$ *and* $\forall j,\ error[j] < \epsilon$ **then**
              $converged \leftarrow$ true;
            end
          end
        end
      end
    end
  end
end

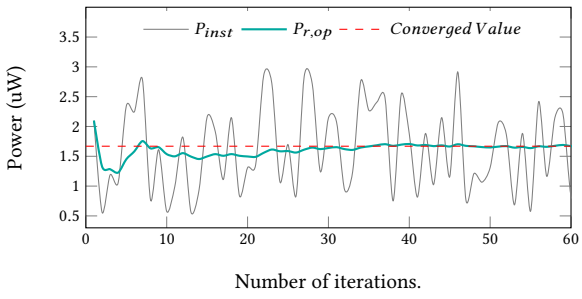

Figure 4: Iterative convergence of the running-mean power estimate for a single resource under one operation.

This condition guarantees that the resource-level power estimate has stabilized and that further simulations would not significantly affect the final average. The resulting converged value $P_{r,op}$ is recorded as the characterized power of resource r when executing operation op in the given neighborhood configuration.

All converged power values are finally stored in a *characterization database* that consolidates the results across all blocks and its resources, operations, and valid neighborhood configurations. Each entry in the database is indexed by a tuple,

$$(s,\ op,\ r,\ N)$$

corresponding respectively to the SiLago block type, the operation being executed, the contributing resource, and the neighborhood context in which characterization was performed. Because each block type supports only a modest number of operations with its limited functional and interconnect resources in the context of its valid neighbours, the dimensionality of this database remains small — typically on the order of tens to a few hundreds of entries for a heterogeneous CGRA fabric. As a result, the database is compact and scalable: adding a new block type increases its size linearly without requiring global re-characterization. For estimation, the required values can be retrieved by directly querying the database.

4 Energy Estimation

Building on the characterization methodology, this section presents the synchoros energy estimation method. As stated earlier, mapping an algorithm as a CGRA design specifies the type, number, and operations implemented by the resources in SiLago blocks. Composition by abutment ensures that the total energy of the CGRA can be expressed as the sum of the energy of its constituent SiLago blocks, eliminating the need for additional correction terms. The total energy of the CGRA that implements an algorithm is,

$$E_{Fabric,m} = \sum_{s \in S} E_{s,m} \tag{4}$$

where S denotes the set of SiLago blocks in the fabric and $E_{s,m}$ is the energy of block s in executing the mapping m, which also identifies the spatio-temporal distribution of operations in the block s.

The energy cost of each block is the sum of contributions from its functional resources, the local NoC segments, and the local clock-tree segment.

$$E_{s,m} = \sum_{r \in R} E_{r,m} + E_{LNoC,m} + E_{LCT,m} \tag{5}$$

Here, $r \in R$ denotes the functional resources, while E_{LNoC} and E_{LCT} account for the local NoC and clock-tree segments in s, respectively.

The energy of functional components is evaluated by accumulating the power in the mapping operations over the number of cycles for each operation:

$$E_{c,m} = \sum_{op \in m} P_{c,op} \cdot n_{c,op} \tag{6}$$

where $P_{r,op}$ is the characterized power, i.e., the converged value of the power of the resource in operation op.

The LNoC and the LCT also exhibit different energy consumption levels depending on their operation. LNoC energy is also implied by the operations, e.g., different energy when sending or receiving data over different source-destination pairs . Their energy is accumulated in the same way as the energy consumed by a resource.

$$E_{LNoC} = \sum_{op \in m} P_{LNoC,op} \cdot n_{LNoC,op} \tag{7}$$

Energy of LNoC segment is different based on its *state*, i.e., when it is active versus when it is gated.

$$E_{LCT} = \sum_{st \in St_{LCT}} P_{LCT,st} \cdot n_{LCT,st} \tag{8}$$

The energy estimation is both accurate and agile. It is *accurate* because the impact of all wires is naturally factored, and no correction terms are required. It is *agile* because all power values are pre-characterized and stored in a database, enabling energy estimation via simple lookups.

5 Experimental Results

In this section, we present the validation results of our methodology, followed by a comparison against two state-of-the-(SOTA)) estimation approaches. We also examine the overheads of synchoricity to assess the trade-offs and benefits of synchoros energy estimations.

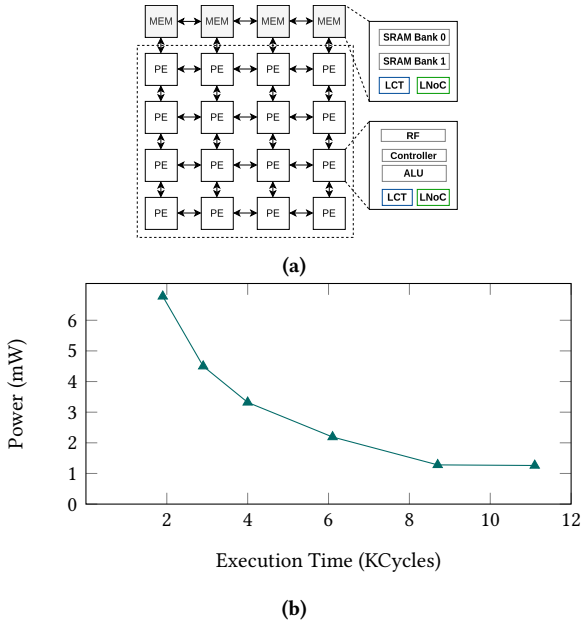

Figure 5: (a) High-level architecture diagram with 4×4 PEs and four memory cells, and (b) pareto points for GEMM mappings, of DRRA and DiMArch CGRA [8, 14, 15].

5.1 Experimental Setup

The energy estimation validation and evaluation are performed on DRRA and DiMArch CGRAs [8, 14, 15] (Fig. 5a). DRRA, or Dynamically Reconfigurable Resource Array, provides computational PEs consisting of a controller, a register file (RF), and an ALU for arithmetic operations. DiMArch, or Distributed Memory Architecture, provides memory cells consisting of SRAM banks to serve as data memory. We choose this CGRA as the increase in computational parallelism in DRRA can be matched with an increase in parallelism in DiMArch. Different architectural alternatives can be implemented by clustering DRRA, DiMArch cells. The number of DRRA columns determines the number of CGRA data memory ports. The cells are physically synthesized and characterized with a convergence criterion of 2%.

Table 1 lists the Berkeley Dwarf [2] and the data sizes of the algorithm workload. Mapping is performed by Vesyla[25], the high-level synthesis tool for DRRA and DiMArch. These mappings are used directly to estimate the energy consumption of the resulting design. To validate the estimates, we sweep the data sizes and all possible combinations of the number of PEs (DRRA cells) and the number of memory cells (DiMArch cells) to construct a Pareto frontier for each algorithm. Six points from each frontier are selected for validation (see Figure 5b). For ground-truth comparison, the corresponding designs are synthesized, placed, and routed in 28nm technology at 100 MHz.

Table 1: Algorithm Specifications.

Algorithm	Berkeley Dwarf [2]	Data Sizes		
		Small	Medium	Large
BACKPROP	Unstructured grids	10^3	10^5	10^6
FFT	Spectral methods	2^8	2^{10}	2^{12}
GEMM	Dense linear algebra	32^2	64^2	128^2
KNN	N-body methods	10^3	10^5	10^6
NW	Dynamic programming	64^2	128^2	256^2
STENCIL	Structured grids	10^3	10^5	10^6

Table 2: CGRA parameters.

Parameter	Range	Rows	Columns
No. of PEs	[4::2::12]	[2::1::6]	[2::1::6]
No. of Memory Cells	[2::1::6]	[1::1::6]	[1::1::6]

5.2 Validation

We validated the total energy consumption and relative energy breakdown for each Pareto design point. Figure 6 shows that our methodology achieves 2% error against ground truth across algorithm variations in functionality, dimensions, and parallelism compared against post-route ground truth results.

Figure 7 shows the relative energy breakdown for a GEMM mapping with medium data size, 4x4 PEs, and 4 memory cells. The impact of all the wires is captured in the characterized energy values and, barring switching data variations, remains unchanged when performing estimations across all resources in the design.

5.3 SOTA Comparison

Accelergy [23] specifies the design with user-defined components that are derived from basic fine-grained components. It focuses on datapath components, but does not consider the inter-component wires. For DRRA and DiMArch, the characterization database is specified in terms of 11 fine-grained components in total. The characterized energy values are prepared from post-layout simulations of independently synthesized fine-grained components. Accelergy has high accuracy at the fine-grained level, such as for adders, exceeding 97% (see Figure 8). However, accuracy decreases at higher granularity because the wires are not accounted for. The impact of wires becomes more significant as the complexity and size of the evaluated micro-architectural units grow.

CGRA-EAM [22] characterizes the power of coarse-grained components after synthesis, which includes more information details compared to the fine-grained approach. However, its accuracy is limited to specific characterized design instances.

5.4 Overheads of synchoricity

Having validated the accuracy of our methodology across functionality, dimensions, and energy breakdown, we now examine the associated overheads of synchoricity. In addition to DRRA+DiMArch

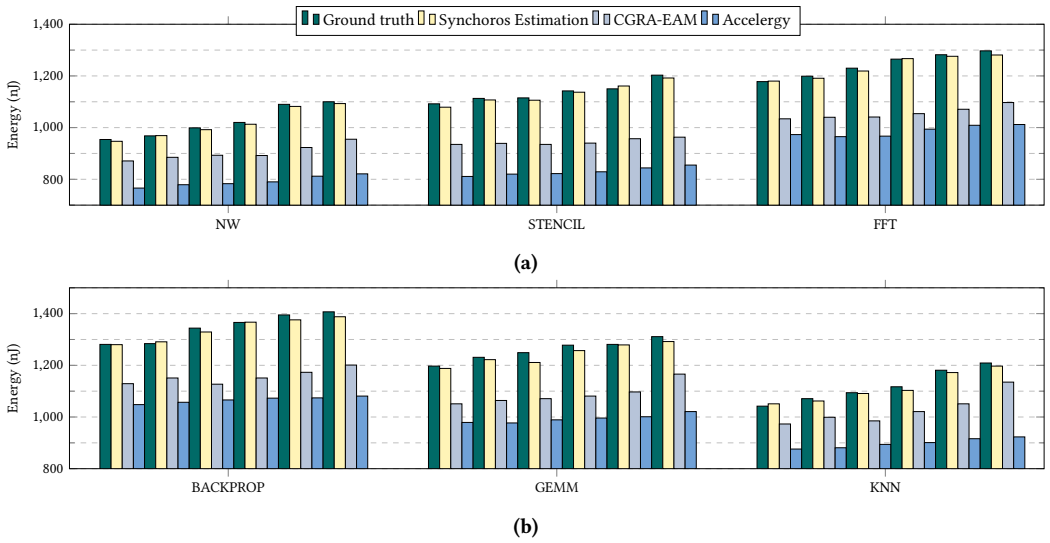

Figure 6: Power estimation comparison with ground truth for three different methodologies.

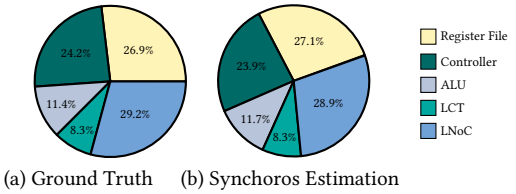

Figure 7: Relative energy breakdown within a DRRA block.

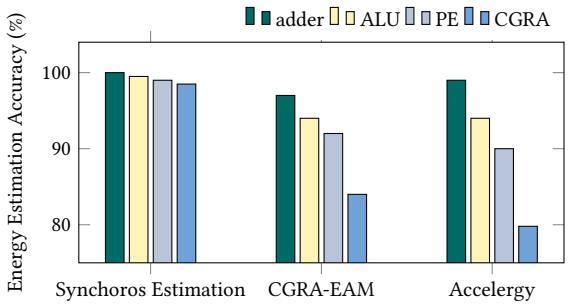

Figure 8: Estimate accuracy at different hierarchical granularity.

CGRAs, we apply synchoros design principles to the HyCUBE CGRA [11] and ADRES CGRA [13] and quantify the design overheads introduced by synchoricity.

To apply synchoricity on designs, the PEs are synthesized and made abutment-ready after fixing the layout and peripheral interconnect locations. We implemented 4x4 CGRAs in 28nm process.

The abutment-ready requirements in synchoricity impose restrictions on the layout of the blocks. Each block has to fit in the virtual grid. The size, shape, and location of the in and out pins are standardized to enable the composition by abutment. Table 3 shows a comparison of additional overhead in terms of total standard cell area and energy with the same switching. Based on the comparison, although synchoricity introduces modest overheads, we believe that the compromises are outweighed by the benefits —namely, agile and accurate energy estimation from higher levels of abstraction. Similar compromises were also adopted when standard cells were introduced and replaced the Mead-Conway full-custom design flow [5].

Design (4x4 PEs)	Std. cell area (μm^2)		Energy (nJ)	
	asynchoros	Synchoros	asynchoros	synchoros
DRRA+DiMArch	92512.6	97786.3	1137.3	1158.7
HyCUBE	87376.4	94055.2	1016.4	1123.9
ADRES	84821.1	90286.1	990.5	1008.8

Table 3: Overhead of synchronicity.

6 Conclusions and Future Work

This paper presented an agile and accurate synchoros energy-estimation methodology for CGRA designs that explicitly accounts for all wires in the fabric. With a one-time characterization effort, the methodology enables post-route-accurate energy estimates at the algorithm level, thereby bridging the gap between algorithm-level estimations and physical-level accuracy.

Follow-up work is planned to extend the methodology to the application level, modeling applications as hierarchies of algorithms and enabling post-route-accurate energy characterization and estimation for an SoC implemented in synchoros VLSI design style.

Acknowledgements

This research was supported by ChipsJU (Chips Joint Undertaking) with the projects Cynergy4MIE and NeAIxt with grant agreement numbers 101140226 and 101194172, respectively.

References

[1] Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Muhammad Shafique. 2019. X-CGRA: An energy-efficient approximate coarse-grained reconfigurable architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2019).

[2] Krste Asanovic, Ras Bodik, Bryan Catanzaro, Joseph Gebis, Parry Husbands, Kurt Keutzer, David Patterson, William Plishker, John Shalf, and Samuel Webb Williams. 2006. The landscape of parallel computing research: A view from berkeley. (2006).

[3] Maxime Henri Aspros, Juan Sapriza, Giovanni Ansaloni, and David Atienza. 2025. A flexible framework for early power and timing comparison of time-multiplexed CGRA kernel executions. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers: Workshops and Special Sessions*.

[4] Wei-Ting J Chan, Pei-Hsin Ho, Andrew B Kahng, and Prashant Saxena. 2017. Routability optimization for industrial designs at sub-14nm process nodes using machine learning. In *ISPD*.

[5] William J Dally, Chris Malachowsky, and Stephen W Keckler. 2013. 21st century digital design tools. In *Proceedings of the 50th Annual Design Automation Conference*.

[6] Graham Gobieski, Ahmet Oguz Atli, Kenneth Mai, Brandon Lucia, and Nathan Beckmann. 2021. Snafu: an ultra-low-power, energy-minimal cgra-generation framework and architecture. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1027–1040.

[7] Jordi Altayó González, Dimitrios Stathis, and Ahmed Hemani. 2021. Synthesis of predictable global NoC by abutment in synchoros VLSI design. In *NOCS*.

[8] Ahmed Hemani, Syed Mohammed Asad Hassan Jafri, and Shayesteh Masoumian. 2017. Synchoricity and NOCs could make billion gate custom hardware centric SOCs affordable. In *Proceedings of the Eleventh IEEE/ACM International Symposium on Networks-on-Chip*.

[9] Victor Huang, Xinkang Chen, Sumeet K Gupta, Azad Naeemi, et al. 2023. A comprehensive modeling platform for interconnect technologies. *IEEE Transactions on Electron Devices* (2023).

[10] Leonardo Rezende Juracy, Alexandre de Morais Amory, and Fernando Gehm Moraes. 2022. A fast, accurate, and comprehensive PPA estimation of convolutional hardware accelerators. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2022).

[11] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In *Proceedings of the 54th Annual Design Automation Conference 2017*. 1–6.

[12] Gabriel H Loh, Samuel Naffziger, and Kevin Lepak. 2021. Understanding chiplets today to anticipate future integration opportunities and limits. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 142–145.

[13] Bingfeng Mei, Mladen Berekovic, and Jean-Yves Mignolet. 2007. Adres & dresc: Architecture and compiler for coarse-grain reconfigurable processors. In *Fine-and coarse-grain reconfigurable computing*. Springer, 255–297.

[14] Dhilleswararao Pudi, Shivam Malviya, Srinivas Boppu, Yu Yang, Ahmed Hemani, and Linga Reddy Cenkeramaddi. 2024. Integer Linear Programming based Simultaneous Scheduling and Binding for SiLago Framework. *IEEE Access* (2024).

[15] Dhilleswararao Pudi, Yu Yang, Dimitrios Stathis, Sunil Kumar Prajapati, Srinivas Boppu, Ahmed Hemani, and Linga Reddy Cenkeramaddi. 2024. Application Level Synthesis: Creating Matrix-Matrix Multiplication Library, A Case Study. *IEEE Access* (2024).

[16] Omar Ragheb, Stephen Wicklund, Matthew Walker, Rami Beidas, Adham Ragab, Tianyi Yu, and Jason Anderson. 2024. CGRA-ME 2.0: A research framework for next-generation CGRA architectures and CAD. In *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*.

[17] Yakun Sophia Shao, Brandon Reagen, Gu-Yeon Wei, and David Brooks. 2014. Aladdin: A pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures. *ISCA* (2014).

[18] Chenlin Shi, Boma Adhi, Shinobu Miwa, and Kentaro Sano. 2024. Post-Route Power Estimation: A Case Study of RIKEN-CGRA. In *2024 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*.

[19] Dimitrios Stathis, Panagiotis Chaourani, Syed MAH Jafri, and Ahmed Hemani. 2023. Clock tree generation by abutment in synchoros VLSI design. *Microprocessors and Microsystems* (2023).

[20] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. 2020. Dsagen: Synthesizing programmable spatial accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE.

[21] Dhananjaya Wijerathne, Zhaoying Li, Manupa Karunaratne, Li-Shiuan Peh, and Tulika Mitra. 2022. Morpher: An open-source integrated compilation and simulation framework for cgra. In *Fifth Workshop on Open-Source EDA Technology (WOSET)*.

[22] Mark Wijtvliet, Henk Corporaal, and Akash Kumar. 2021. CGRA-EAM—rapid energy and area estimation for coarse-grained reconfigurable architectures. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* (2021).

[23] Yannan Nellie Wu, Joel S Emer, and Vivienne Sze. 2019. Accelergy: An architecture-level energy estimation methodology for accelerator designs. In *ICCAD*.

[24] Zhiyao Xie, Rongjian Liang, Xiaoqing Xu, Jiang Hu, Yixiao Duan, and Yiran Chen. 2021. Net2: A graph attention network method customized for pre-placement net length estimation. In *ASP-DAC*.

[25] Yu Yang and Ahmed Hemani. 2022. Vesyla-ii: An algorithm library development tool for synchoros vlsi design style. *arXiv preprint arXiv:2206.07984* (2022).